1주차 발표 스터디 노트: 캡슐화와 서버 바인딩

Oct 19, 2026 · @유진영

발표 장표 순서대로, 택배와 집 비유로 풀었습니다. 전문 용어는 처음 나올 때 괄호로 풀었고, 장표마다 "한 줄로"를 안 보고 말할 수 있으면 준비 끝입니다.

비유 사전

문서 전체에서 이 비유를 계속 씁니다. 발표에서도 그대로 써도 좋습니다.

비유	실제 이름	하는 일
쪽지	HTTP 메시지	"로그인 페이지 주세요" 같은 실제 부탁 내용
상자와 번호표	TCP 헤더	몇 번 창구(포트)로 갈지, 몇 번째 상자인지
송장	IP 헤더	받는 집 주소와 보내는 집 주소
트럭 운행표	이더넷 헤더	이번 트럭이 갈 바로 다음 정거장(MAC 주소)
택배 터미널	라우터, 공유기	송장을 보고 다음 터미널로 보냄
집의 문	IP 주소 (내 컴퓨터가 가진 것)	안방 문 127.0.0.1, 현관 192.168.x.x 등
별채	Docker 컨테이너	집 안에 따로 지은 작은 집. 문도 따로 있음

캡슐화: 택배를 격겹이 싸는 일 (장표 2~3)

장표 2 · 요청 하나에 헤더가 네 겹 붙는다

한 줄로: 쪽지(HTTP)를 상자(TCP)에 넣고, 송장(IP)을 붙이고, 트럭(이더넷)에 싣는다. 받는 쪽은 거꾸로 푼다.

브라우저가 "이 페이지 주세요"라는 쪽지를 씁니다. 이 쪽지를 그대로 던질 수는 없으니, 보내기 전에 네 번 포장합니다. 포장할 때마다 겨에 종이(헤더) 한 장을 붙이고, 그 종이에는 그 단계에서 필요한 정보만 적습니다. 포장하는 사람은 안에 뭐가 들었는지 열어 보지 않습니다.

- 이렇게 싸는 것을 **캡슐화**, 받는 쪽에서 하나씩 푸는 것을 **역캡슐화**라고 합니다.
- 포장 단계마다 부르는 이름이 다릅니다: 쪽지만 있으면 **메시지**, 상자에 넣으면 **세그먼트**, 송장을 붙이면 **패킷**, 트럭에 실으면 **프레임**. (책 01-3)
- 트럭에는 짐칸 끝에 파손 검사표(FCS)도 붙습니다. 가는 도중 짐이 상하면 받는 쪽이 알아차리고 버립니다. 그래서 이더넷만 앞뒤를 둘 다 감쌉니다.

이런 질문을 받으면

- *와이파이면 이더넷이 아니지 않나?* 트럭 대신 오토바이로 배달하는 것뿐입니다. 맨 바깥에서 한 구간만 맡는 자리는 같아요. (책 07-3)
- *HTTPS면?* 쪽지를 자물쇠 달린 상자(TLS)에 한 번 더 넣습니다. 중간에 있는 사람은 송장은 볼 수 있지만 쪽지는 못 읽습니다.
- *포장이 너무 많아 낭비 아닌가?* 포장지는 다 합쳐 수십 바이트로 작습니다. 대신 단계를 나눈 덕분에 트럭을 오토바이로 바꾸든(유선→와이파이) 쪽지 양식을 바꾸든(HTTP/1.1→HTTP/2) 나머지는 그대로 쓸 수 있습니다.

장표 3 · 가장 짧은 구간을 맡는 계층이 가장 바깥에 붙는다

한 줄로: 트럭은 터미널마다 갈아타지만 송장은 끝까지 같다. 그래서 자주 갈아 끼우는 트럭 운행표가 맨 바깥에 있다.

서울에서 부산으로 택배를 보내면 동네 → 서울 터미널 → 대전 터미널 → 부산으로 가고, 구간마다 트럭이 다릅니다. 인터넷도 같습니다. 터미널(라우터)은 패킷을 받을 때마다 이렇게 합니다.

1. 트럭에서 짐을 내린다. (이더넷 포장을 벗긴다)
2. 송장의 받는 주소를 보고 다음 터미널을 정한다. (IP 헤더를 보고 라우팅 테이블을 찾는다, 책 03-3)
3. 다음 터미널이 어느 트럭 정류장인지 알아낸다. (ARP로 다음 장비의 MAC 주소를 알아낸다, 책 03-1)
4. 새 트럭에 싣고 보낸다. (새 이더넷 포장을 씌운다)

이 과정에서 상자(TCP)와 쪽지(HTTP)는 열어 보지도 않습니다. 만약 송장이 트럭 운행표보다 바깥에 있다면, 트럭을 갈아탈 때마다 송장까지 떼었다 붙여야 합니다. 그래서 **자주 바뀌는 포장일수록 바깥**이라고 기억하면 순서를 외울 필요가 없습니다.

작은 정정: 송장도 아주 조금은 바뀝니다. 터미널을 지날 때마다 "남은 경유 횟수"(TTL)가 1씩 줄고, 0이 되면 버려져서 패킷이 길을 잃고 영원히 돌지 않게 합니다. 보내는 집 주소는 공유기에서 바뀝니다(NAT, 주영님 발표). 받는 집 주소는 끝까지 그대로입니다.

이런 질문을 받으면

- *공유기도 라우터인가?* 네. 공유기는 동네 터미널에 여러 기능(NAT, 와이파이 등)을 합친 상자입니다.
- *스위치는?* 같은 동네 안에서 집을 찾아 주는 배달원입니다. 트럭을 갈아태우지는 않습니다. (책 02-4)
- *짐이 너무 크면?* 트럭 한 대에 실을 수 있는 크기(이더넷은 보통 1500바이트, MTU)가 정해져 있어서 여러 상자로 나눠 보냅니다. (책 03-1)

주소와 로드밸런서 (장표 4~5)

장표 4 · 겹마다 가리키는 대상이 다르다

한 줄로: 바깥 포장일수록 가까운 곳을, 안쪽 포장일수록 더 구체적인 곳을 가리킨다.

아파트에 사는 친구에게 부탁을 전한다고 생각해 보세요.

포장	주소	비유
이더넷	MAC 주소	지금 짐을 넘겨받을 바로 옆 사람
IP	IP 주소	친구가 사는 아파트 동·호수
TCP	포트 번호	그 집에 사는 사람 중 누구 (컴퓨터 안의 프로그램)
HTTP	Host와 경로	그 사람에게 부탁하는 구체적인 물건

- 포트는 0부터 65535까지의 번호입니다. 서버 프로그램은 정해진 번호에서 기다리고, 브라우저 쪽 번호는 운영체제가 그때그때 빈 번호를 골라 줍니다. (책 04-1)
- 컴퓨터 한 대는 IP 주소를 여러 개 가질 수 있습니다. 이게 장표 7의 "집의 문" 이야기로 이어집니다.

이런 질문을 받으면

- *서버는 443번 하나로 수천 명을 어떻게 구분하나?* 같은 우체통에 편지가 와도 보낸 사람 주소가 다르면 구분되는 것과 같습니다. 연결은 보낸 쪽 IP·포트와 받는 쪽 IP·포트 네 가지를 함께 봐서 구분합니다. 자세한 건 주영님 발표에서 나옵니다.
- *MAC 주소는 기기마다 고유한데 그걸로 인터넷을 하면 안 되나?* MAC은 주민번호처럼 고유하지만 어디 사는지는 알려 주지 않습니다. IP는 주소라서 "서울 쪽으로 보내면 된다" 같은 길 찾기가 가능합니다. (책 03-2)

장표 5 · 로드밸런서의 L4·L7이 바로 이 계층 번호다

한 줄로: L4는 송장과 창구 번호만 보고 나눠 주는 분류원, L7은 상자를 열어 쪽지까지 읽고 나눠 주는 분류원이다.

로드밸런서는 서버 여러 대 앞에서 요청을 나눠 주는 분류원입니다. L4·L7의 숫자는 OSI 7계층의 번호로, 4는 TCP가 있는 층, 7은 HTTP가 있는 층입니다.

	L4 분류원	L7 분류원
보는 것	IP와 포트 (송장과 창구 번호)	HTTP 내용까지 (쪽지)
잘하는 것	빠르고 단순하게 나눠 주기	"/api는 A 서버, 이미지는 B 서버"처럼 내용을 보고 나눠 주기
HTTPS일 때	자물쇠 상자를 열지 않고 그대로 넘길 수 있음	열쇠(인증서)가 있어야 쪽지를 읽을 수 있음
예	AWS NLB	AWS ALB, nginx

이런 질문을 받으면

- *nginx는 어느 쪽?* 둘 다 할 수 있습니다. 보통 쓰는 `http` 설정은 L7, `stream` 설정은 L4입니다.

- *WebSocket은?* 둘 다 통과할 수 있습니다. L7에서는 "프로토콜을 바꾸겠다"는 요청을 제대로 넘기도록 설정해야 하고, 이건 6주차 주제입니다.
- *L7이 자물쇠를 푸는 건 괜찮나?* 분류원과 서버 사이에서는 상자가 열린 채로 갈 수 있습니다. 필요하면 다시 잠그고 보내기도 합니다. 우리 구성은 어느 쪽인지 확인해 두면 좋습니다.

서버 바인딩 (장표 6~8)

장표 6 · 127.0.0.1에서 돌던 서버는 왜 밖에서 안 보일까

화제를 바꾸는 장표입니다. "Docker로 띄운 서버가 브라우저에서 안 열린 적 있나요?", "휴대폰으로 내 노트북 서버에 못 들어가 본 적 있나요?" 하고 물어보며 시작하세요. 두 경우의 원인이 같다고 미리 말해 두면 듣는 사람이 바로 따라옵니다.

장표 7 · 바인딩 주소가 어떤 요청을 받을지 정한다

한 줄로: 컴퓨터에는 문이 여러 개 있고, 서버는 그중 어느 문 앞에서 손님을 기다릴지 고른다.

집에 문이 여러 개 있다고 생각해 보세요. 안방 문(127.0.0.1)은 집 안에 있는 사람만 쓸 수 있고, 현관(와이파이에서 받은 192.168.x.x)은 동네 사람도 두드릴 수 있습니다. 서버 프로그램은 켜질 때 "어느 문의 몇 번 창구에서 기다릴지"를 정하는데, 이걸 **바인딩**이라고 합니다. 손님이 기다리는 사람이 없는 문을 두드리면 아무도 열어 주지 않고, 브라우저에는 "연결 거부(connection refused)"가 뜹니다.

- **127.0.0.1 = 안방 문**: 밖에서 온 손님은 이 문에 아예 닿을 수 없습니다. 이름은 루프백입니다. (책 03-2 [좀 더 알아보기])
- **0.0.0.0 = "모든 문에서 기다려"**: 바인딩할 때만 쓰는 표기입니다. 손님이 찾아가는 주소가 아니에요. IPv6에서는 `::`가 같은 뜻입니다.
- **특정 주소 = 문 하나만**: 문이 많은 서버에서 "내부망 문으로 온 손님만 받기" 같은 용도로 씁니다.

이런 질문을 받으면

- *localhost랑 127.0.0.1은 같은 거죠?* 거의 같지만 완전히는 아닙니다. localhost는 문의 "이름"이라 IPv6 안방 문(`::1`)을 가리킬 수도 있습니다. 서버는 127.0.0.1에서 기다리는데 손님은 `::1`로 가서 못 만나는 일이 실제로 있습니다.
- *브라우저에 0.0.0.0:3000을 치면?* 환경에 따라 열리기도 하지만 원래 쓰는 법이 아닙니다. localhost나 실제 IP를 쓰세요.

장표 8 · 컨테이너 안의 127.0.0.1은 컨테이너 자신이다

한 줄로: 컨테이너는 마당에 지은 별채다. 별채의 안방 문은 본채의 안방 문과 이름만 같고 다른 문이다.

장표의 그림을 별채 비유로 따라가 보면 이렇습니다.

1. 본채(내 컴퓨터)의 브라우저가 본채 3000번 창구를 두드린다.

2. `-p 3000:3000`은 "본채 3000번으로 온 손님은 별채 현관(eth0)의 3000번으로 안내해"라는 뜻입니다. 앞 숫자가 본채, 뒤 숫자가 별채입니다.
3. 그런데 별채 안의 서버는 별채 안방 문(127.0.0.1)에서만 기다리고 있다.
4. 별채 현관에는 아무도 없으니 손님은 돌아간다. 브라우저에는 ERR_CONNECTION_REFUSED나 ERR_EMPTY_RESPONSE 같은 오류가 뜹니다.

해결은 별채 서버도 **모든 문(0.0.0.0)에서 기다리게** 하는 것입니다. 그러면 현관으로 온 손님도 만납니다.

내가 처음 고른 답 "3000번은 예약 포트"가 틀린 이유도 말할 수 있어야 합니다. 0~1023번은 유명한 가게 전용 번호(80 웹, 443 HTTPS, 22 SSH)이고, 3000번은 아무나 쓰는 번호입니다. (책 04-1)

이런 질문을 받으면

- *Mac의 Docker Desktop도 같나?* 네. Mac에서는 별채가 다시 가상 건물 안에 있어서 한 단계가 더 있을 뿐, 별채 안방 문이 별채 것이라는 점은 같습니다.
- *별채에서 본채의 DB에 localhost로 접속하면?* 같은 이유로 안 됩니다. localhost가 별채 자신을 가리키니까요. Docker Desktop에서는 `host.docker.internal`, compose에서는 서비스 이름(예: `db`)으로 접속합니다.
- `--network host`*는?* 리눅스에서 별채와 본채 사이 벽을 허문 것과 같습니다. 편하지만 따로 떼어 둔 의미가 사라져서, 개발할 때가 아니면 권하지 않습니다.

확인, 보안, 정리 (장표 9~11)

장표 9 · 확인하고 고치는 법

한 줄로: 서버가 어느 문 앞에서 기다리는지는 명령 하나로 바로 볼 수 있다.

```
# macOS (다른 사용자 프로그램까지 보려면 앞에 sudo)
lsof -iTCP -sTCP:LISTEN -n -P

# Linux
ss -ltnp
```

"지금 손님을 기다리는(LISTEN) 프로그램을 보여 줘"라는 뜻입니다. 결과에서 주소 부분만 읽으면 됩니다.

보이는 것	뜻
`127.0.0.1:3000`	안방 문에서만 기다림. 이 컴퓨터 안에서만 접속됨
`*:3000` 또는 `0.0.0.0:3000`	모든 문에서 기다림. 다른 기기에서도 접속됨
`[::1]:3000`	IPv6 안방 문

보이는 것	뜻
`[::]:3000`	IPv6 모든 문

장표의 도구별 기본값은 한 문장으로 말하면 됩니다. "개발 도구는 대부분 안전하게 안방 문이 기본이고, Node는 주소를 안 주면 모든 문에서 기다린다." 버전마다 기본값이 달라질 수 있으니, 헷갈리면 위 명령으로 확인하는 게 가장 확실합니다.

장표 10 · 0.0.0.0은 누구의 요청이든 받는다는 뜻이다

한 줄로: 모든 문을 열어 두면 친구도 들어오지만 모르는 사람도 들어온다.

- **카페 와이파이**: 같은 카페에 있는 사람은 내 노트북 현관을 두드릴 수 있습니다. 비밀번호 없는 개발 서버나 Redis가 모든 문에서 기다리고 있으면 그대로 보입니다. macOS가 "들어오는 연결을 허용할까요?" 창을 띄울 때 생각 없이 허용하면 문이 열립니다.
- **Docker `-p` 의 기본**: 주소를 안 적으면 본채의 모든 문에 창구를 엽니다. 리눅스에서는 Docker가 경비원(방화벽 ufw)보다 먼저 문 규칙을 써 버려서, 경비원이 막아 둔 문이 열려 있을 수 있습니다. "ufw로 막았는데 밖에서 DB에 붙는" 사고가 이렇게 생깁니다.
- **해법**: 내 컴퓨터에서만 쓸 창구는 안방 문을 적습니다. `-p 127.0.0.1:5432:5432`, compose에서는 `ports: ["127.0.0.1:5432:5432"]`. 별채끼리만 쓰는 DB라면 `ports`를 아예 빼도 됩니다. 같은 compose 안에서는 서비스 이름으로 찾아갈 수 있어요.

팀 규칙으로 제안하기 좋은 지점입니다. 예: "밖에 공개할 필요 없는 포트는 compose에 127.0.0.1을 붙인다."

장표 11 · 정리

네 줄을 하나씩 읽기보다 두 문장으로 묶어서 말하면 좋습니다. "자주 바뀌는 포장일수록 바깥에 있다. 서버는 자기가 고른 문으로 온 손님만 받는다." 그다음 "우리 서비스에서는" 박스를 실제로 확인한 결과로 채워서 말합니다.

- [ ] 우리 docker compose 파일에서 DB·Redis 등의 `ports`가 127.0.0.1로 묶여 있는지 확인
- [ ] 스테이징·운영 서버에서 `ss -ltnp`로 모든 문(0.0.0.0)에 열린 포트 목록 뽑아 보기
- [ ] 확인 결과를 장표 11의 대괄호 자리에 채우기

발표 전에 해 볼 데모

장표 9에서 3분만 보여 주면 "안방 문"과 "모든 문"의 차이가 눈으로 보입니다. 발표 전에 한 번 돌려 보세요.

1. **안방 문에서 기다리게 하기**: 노트북에서 `python3 -m http.server 8000 --bind 127.0.0.1` 실행

2. **확인**: 다른 터미널에서 `lsof -iTCP -sTCP:LISTEN -n -P | grep 8000` → `127.0.0.1:8000`이 보임
3. **내 노트북 현관 주소 알아내기**: macOS `ipconfig getifaddr en0`, Linux `hostname -I`
4. **밖에서 두드려 보기**: 같은 와이파이의 휴대폰으로 `http://<노트북 IP>:8000` → 안 열림
5. **모든 문에서 기다리게 바꾸기**: 서버를 끄고 `--bind 0.0.0.0`으로 다시 실행 → lsof에 `*:8000` → 휴대폰에서 열림
6. **(선택) 별채에서 똑같이**: `docker run --rm -p 8000:8000 python:3.12-slim python -m http.server 8000 --bind 127.0.0.1` 후 `curl localhost:8000` → 실패. `--bind 0.0.0.0`으로 바꾸면 성공

미리 알아 둘 것:

- 회사 와이파이는 기기끼리 서로 못 보게 막아 둔 경우가 있어서 5번에서도 안 열릴 수 있습니다. 미리 해 보고, 안 되면 휴대폰 핫스팟에 노트북을 붙여서 하세요.
- macOS가 방화벽 허용 창을 띄우면 데모 동안만 허용하고, 끝나면 서버를 꺼 두세요. 장표 10의 위험을 직접 보여 주는 장면이기도 합니다.
- 6번은 이미지를 내려받아야 하니 `docker pull python:3.12-slim`을 미리 해 두세요.

발표 끝나고 받을 만한 질문

- *0.0.0.0으로 두면 인터넷에 있는 모든 사람이 들어오나?* 내 집까지 올 수 있는 사람만입니다. 공유기 뒤에 있는 노트북이라면 보통 같은 와이파이까지입니다. 클라우드 서버처럼 인터넷에 바로 붙은 기계라면 보안 그룹과 방화벽이 마지막 대문이고, 그게 열려 있으면 정말 누구나 들어옵니다.
- *운영 서버의 앱은 어느 문에 두나?* nginx가 같은 서버에 있다면 nginx가 현관을 지키고, 앱은 안방 문(127.0.0.1)에 둡니다. 컨테이너라면 별채 안에서는 모든 문에서 기다리되, 본채에 여는 창구(`-p`)는 필요한 것만 주소를 적어 엽니다.
- *별채 안에서 0.0.0.0이면 위험하지 않나?* 별채의 모든 문일 뿐입니다. 밖에서 별채까지 올 수 있는지는 본채의 `-p` 설정이 정하니, 지켜야 할 곳은 `-p` 쪽입니다.

더 읽을거리

- 《혼자 공부하는 네트워크》 01-3 캡슐화와 역캡슐화, PDU / 02-1 이더넷 프레임 / 03-1 ARP / 03-2 [좀 더 알아보기] 예약 주소: 0.0.0.0 vs 127.0.0.1 / 03-3 라우팅 테이블 / 04-1 포트의 분류
- Docker 문서: Packet filtering and firewalls: Docker와 ufw·iptables의 관계
- Docker 문서: Port publishing: `-p`의 주소 지정 형식

이 문서의 도구별 기본값과 Docker 방화벽 내용은 공식 문서를 열어 확인하지 못한 채로 작성했습니다. 발표에서 단정하기 전에 위 두 문서와 데모로 한 번 확인해 주세요.